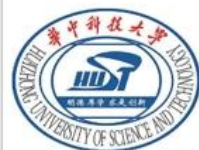


软件工程

第4章 软件设计

4.2 体系结构设计(第2部分: 体系结构精化)



回顾(2): 如何设计软件体系结构?

辩识关键需求(关键的功能需求和关键的质量需求)

创建体系结构雏形(选取适当的模式, 注入分析模型中的职责, 建立逻辑关联)

精化体系结构

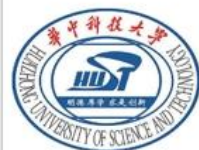
验证体系结构

体系结构精化的目标

- 将体系结构雏形精化成为全面的、设计适度的、优化的软件体系结构

软件资产：现成的软件系统/子系统，开发机构自身积累的构件库，业界广泛采用的类库、中间件或框架，以及Internet上开源的软件项目，等

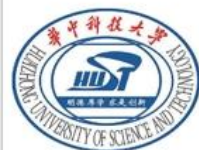
如何达成上述目标？



如何获得全面的软件体系结构?

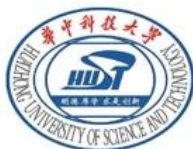
□ 以体系结构雏形为基础，以需求为目标，逐项需求对当前体系结构进行质疑：

- ◆ 基于当前体系结构是否能够实现该需求项？如果不能，如何修改当前体系结构？
- ◆ 首先针对关键需求项，然后针对一般需求项



如何获得设计适度的软件体系结构？

- 针对当前体系结构中的每个模块进行质疑：
 - ◆ 职责定义是否清晰？
 - ◆ 对外接口定义是否清晰？ 服务请求接口是否与服务提供接口精确匹配？
 - ◆ 接口定义是否与职责定义匹配？ 即，模块的服务提供接口中的功能是否完整地涵盖其职责？

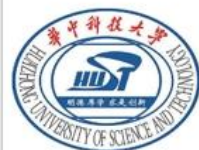


如何获得优化的软件体系结构？

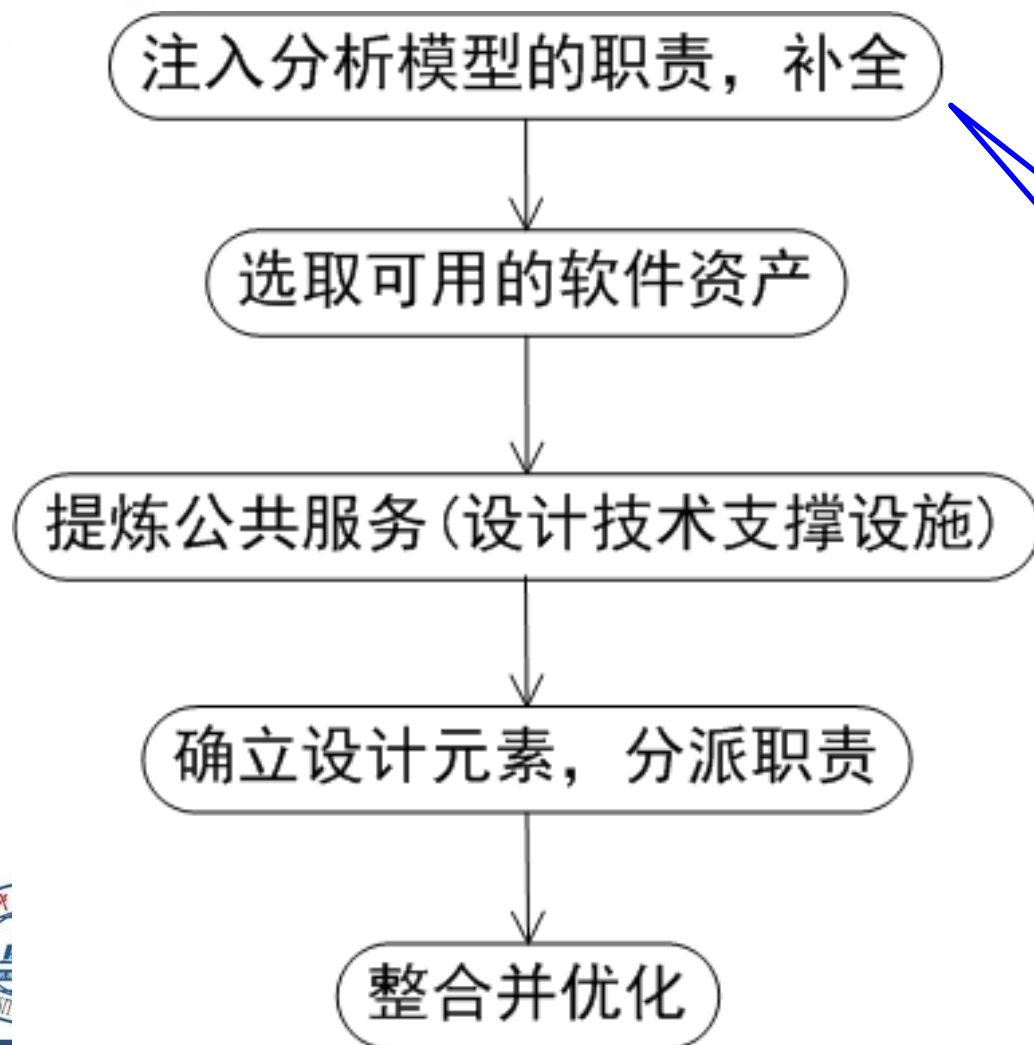
- 针对当前体系结构进行质疑：
 - ◆ 职责是否可以利用现有的设计资产来实现？
 - ◆ 多项职责的实现是否可以共用相同的基础服务？
- 逐项需求对当前体系结构进行质疑：
 - ◆ 是否存在更合理、更优化的实现方式？
 - ◆ 为了更合理、更优化地实现该需求项，是否需要调整当前体系结构？

小结：如何精化软件体系结构？

- 以体系结构雏形为基础
- 以需求的优化实现为目标
- 质疑驱动
- 迭代精化



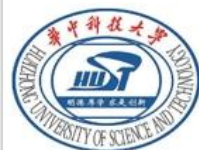
根据体系结构精化的目标和任务， 导出其过程模型



如何做？
为什么？

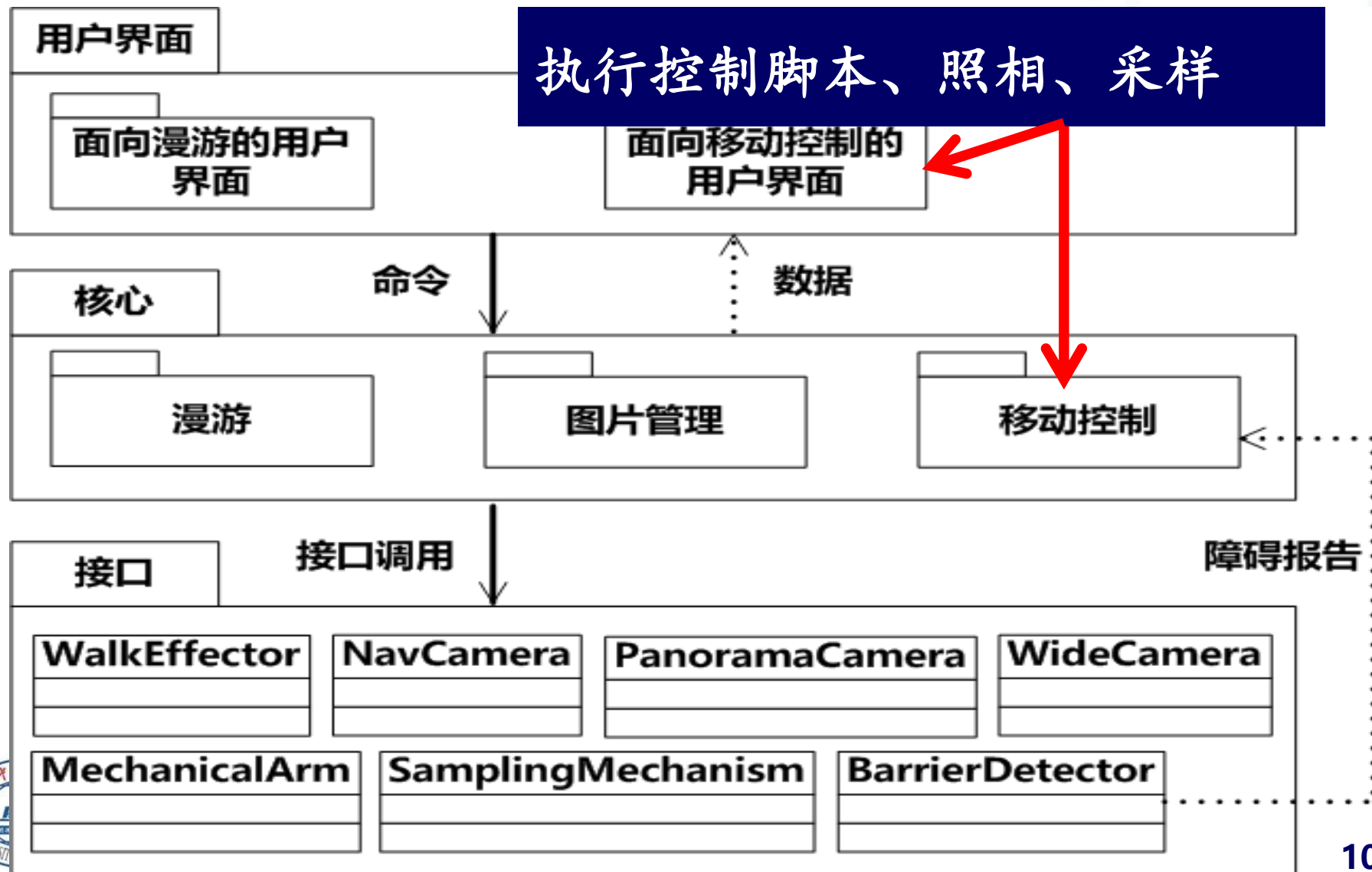
如何“注入分析模型的职责并补全”？

- 逐项针对功能需求项，将分析模型中的相关职责组织为模块(包、构件、子系统或类)，注入至当前体系结构
 - ◆ 先关键需求项，后一般需求项
- 为什么首先要注入分析模型的职责并补全？
 - ◆ 后续步骤必须综合考虑所有职责

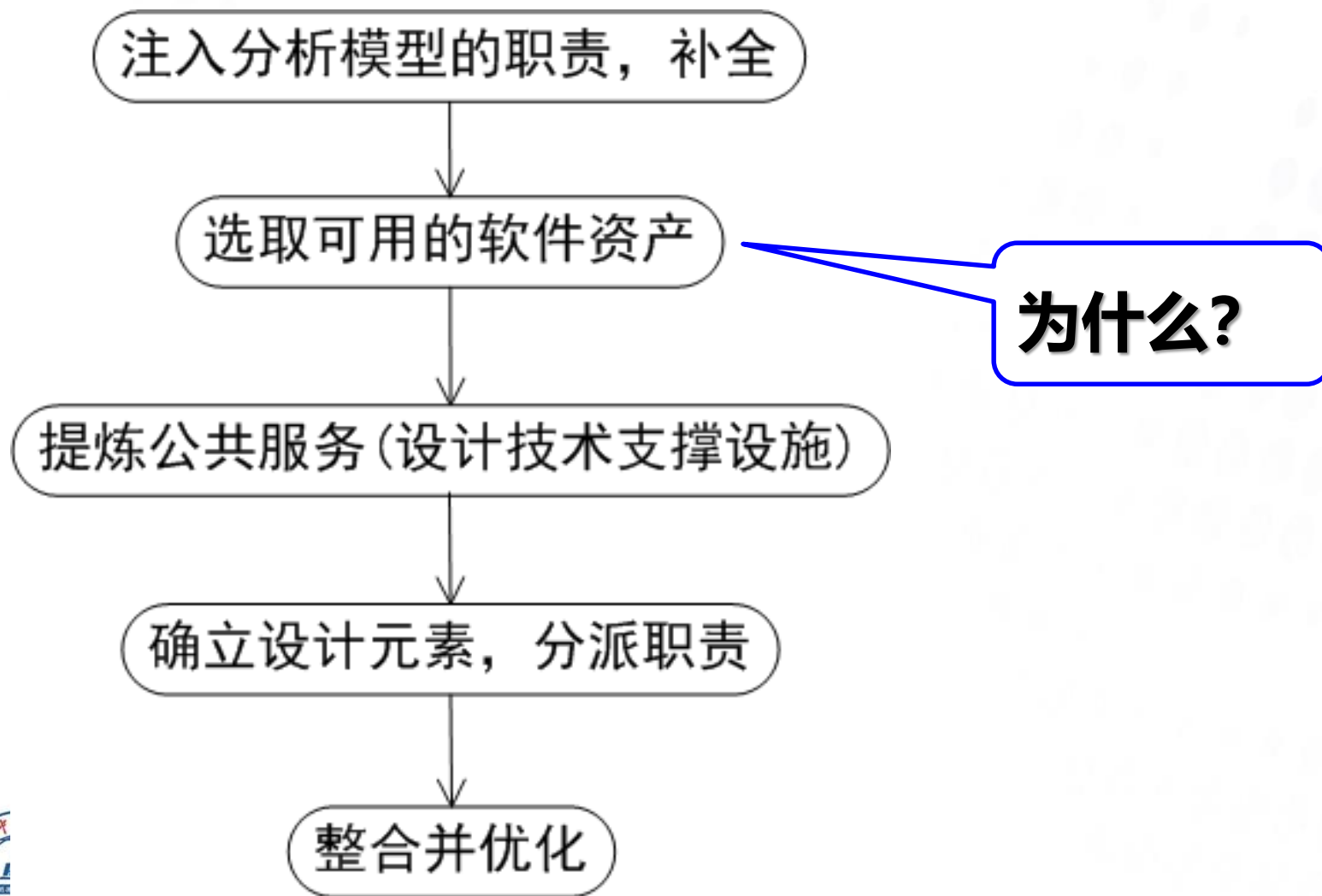


案例分析：MCS体系结构的补全

执行控制脚本、照相、采样



体系结构精化的过程模型(2)

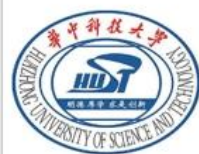


为什么要“选取可用的软件资产”？

- 关键意图：在后续的设计或精化过程中通过适当地调整现有的体系结构来采纳、融入现有的可复用的软件资产
- 为什么？
 - ◆ 复用远比从零开始构造要简单、高效、节约得多
 - ◆ 软件资产已经过大量实践检验，复用它们可显著提高目标软件的质量

为什么要在“提炼公共服务”之前 “选取可用的软件资产”？

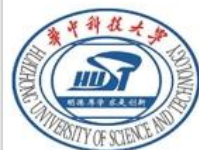
- 如果已有现成的可复用软件资产，没有必要再设置“公共服务”以支撑分析职责的实现



如何“选取可用的软件资产”？

□ 逐项针对分析模型中的职责，研究现有的软件资产：

- ◆ 不可复用：忽略
- ◆ 可直接复用：清晰地定义资产与目标软件系统之间的交互接口，包括数据交换的格式或互操作协议
- ◆ 修改后可复用：怎么办？



如何调整以复用软件资产？

□ 两种调整方法：

- ◆ 修改当前体系结构中面向可复用软件资产的服务请求接口，使之与其提供的服务接口相匹配
- ◆ 如果上述直接修改方法会导致不良影响，则采用适配器方法（容后详述）

案例分析：在MCS的体系结构中 纳入可复用的软件资产

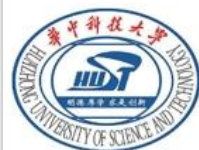
- 假定已有构件“LandscapeRender”可将3维地图数据呈现为二维图像，其接口为：

```
void preRender(LandscapeData, imgSize)
```

```
Image renderLandscape(scale, center)
```

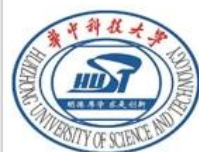
```
void postRender()
```

但在MCS中地图数据的类型为MapData，怎么办？



为复用LandscapeRender而调整接口

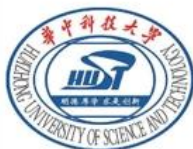
- 方案1：直接将分析模型中的MapData类调整为LandscapeData类
 - ◆ 优点：实现简单、高效
 - ◆ 缺点：与外部构件的耦合度增高，损害可替换性、可修改性
- 方案2：设置新的设计类Map2LandscapeData，将MCS中MapData转换为LandscapeData
 - ◆ 优点：可替换性、可修改性较好
 - ◆ 缺点：损害性能、内存开销加大



为复用LandscapeRender而调整体系结构雏形

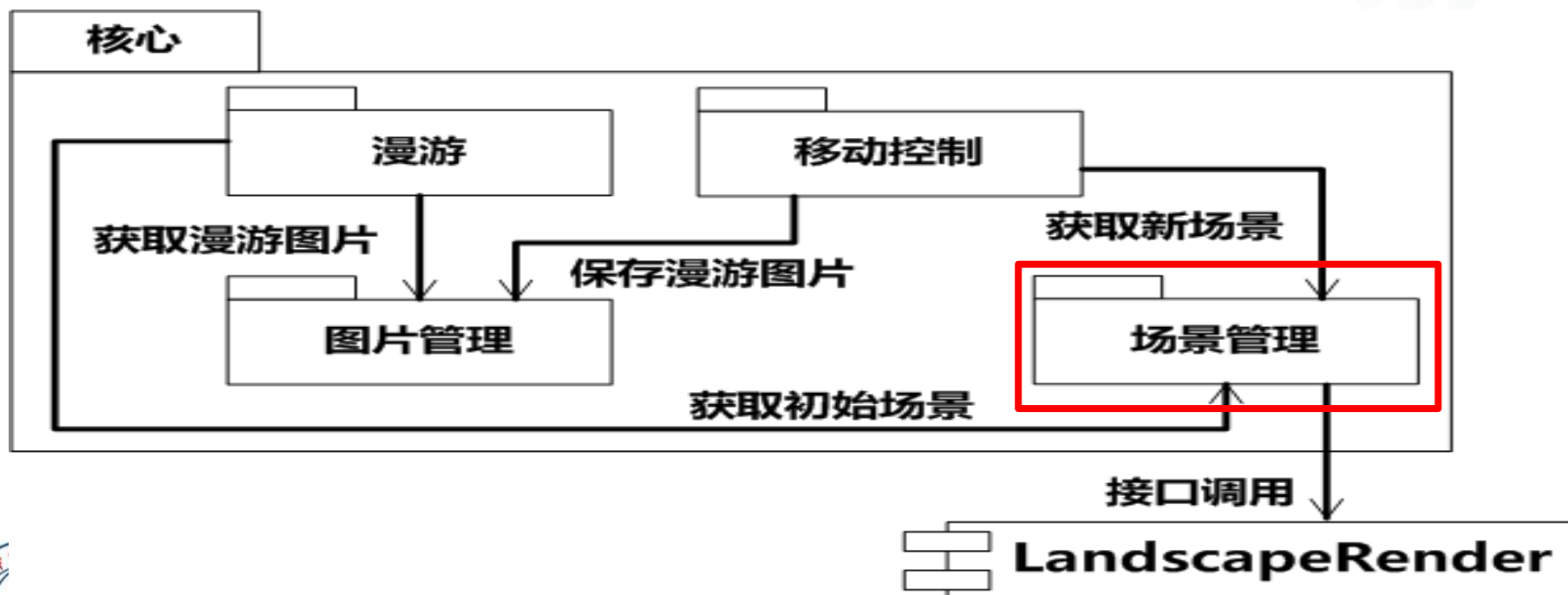
- ❑ 方案A：分析模型中与场景呈现相关的职责
(McsManager的buildSceneByMapData和
CmdExecutor的adjustScene)分别负责地图数据的转换，分别调用LandscapeRender构件的接口
- ❑ 方案B：设置新模块“场景管理”承接上述两项职责，
MCS中仅该模块与LandscapeRender构件交互

选取哪种方案？

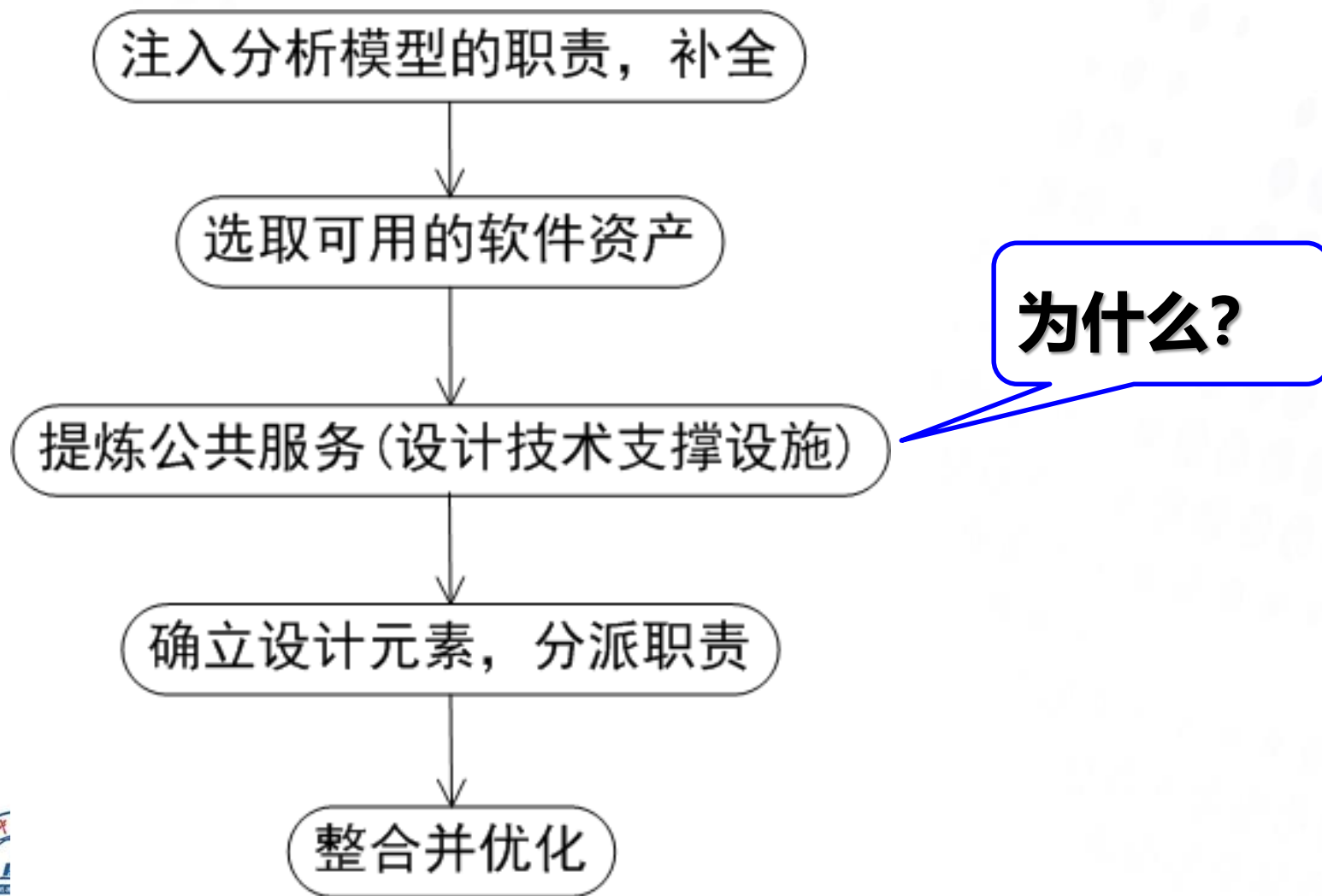


选取方案2和B的组合

- 设计决策的理由1：月面地图数据只需一次性加载，性能开销可接受，同时假设内存空间足够
- 理由2：强调MCS的可替换性、可修改性



体系结构精化的过程模型(3)

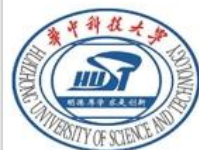


为什么要“提炼公共服务”？

- 关键意图：将为实现分析模型中多项职责所必需的公共性、基础性的服务功能抽取出来，作为整个目标软件系统的技术支撑设施
- 为什么？
 - ◆ 基础服务功能“一次实现、多次调用”

为什么要在“确立设计元素”之前 “提炼公共服务”？

- 便于设计元素直接使用公共服务
- 便于思考如何以公共服务支持分析模型中职责的实现



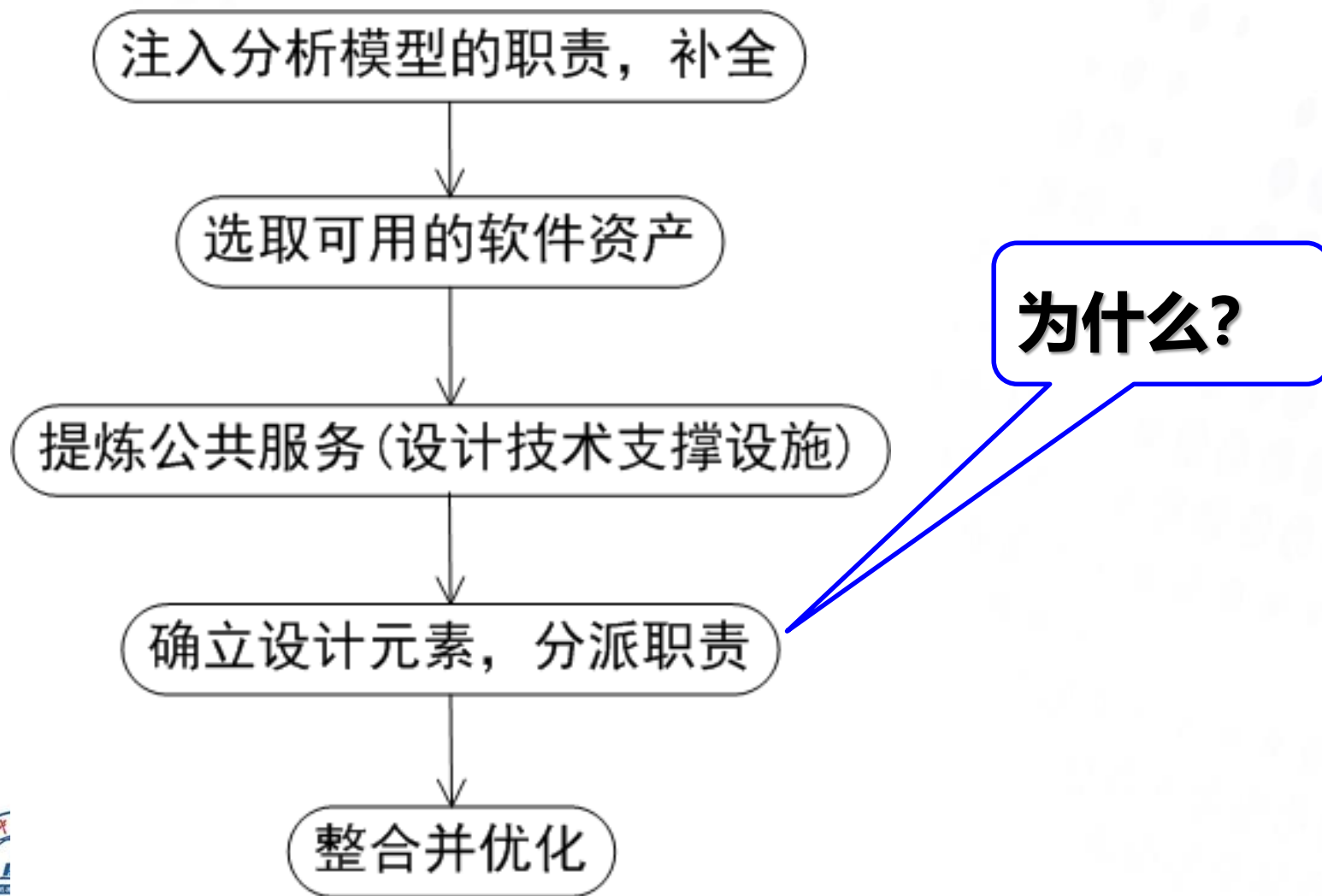
如何“提炼公共服务”？

- 将分析模型的职责中具有全局性（或在一定范围内具全局性）的公共子功能抽取为公共服务
 - ◆ 类似于程序设计中的公共子函数提取
 - ◆ 例如 安全控制服务
- 研究目标软件在与运行环境（操作系统、数据库管理系统等）交互时必要的公共服务
 - ◆ 例如 数据的存储与访问服务

案例分析：针对MCS提炼公共服务



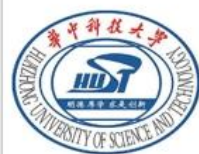
体系结构精化的过程模型(4)



何谓“确立设计元素”？

□ 关键意图：

- ◆ 针对分析模型中的单项或多项职责，采用何种设计元素（子系统、软构件、包还是类）来承担此职责？
- ◆ 如何定义其对外接口以协同工作？
- ◆ 是否有必要及如何采用前面获得的软件资产和公共服务？



为什么要“确立设计元素”？

□ 从需求分析模型迈向软件设计模型的必要步骤：

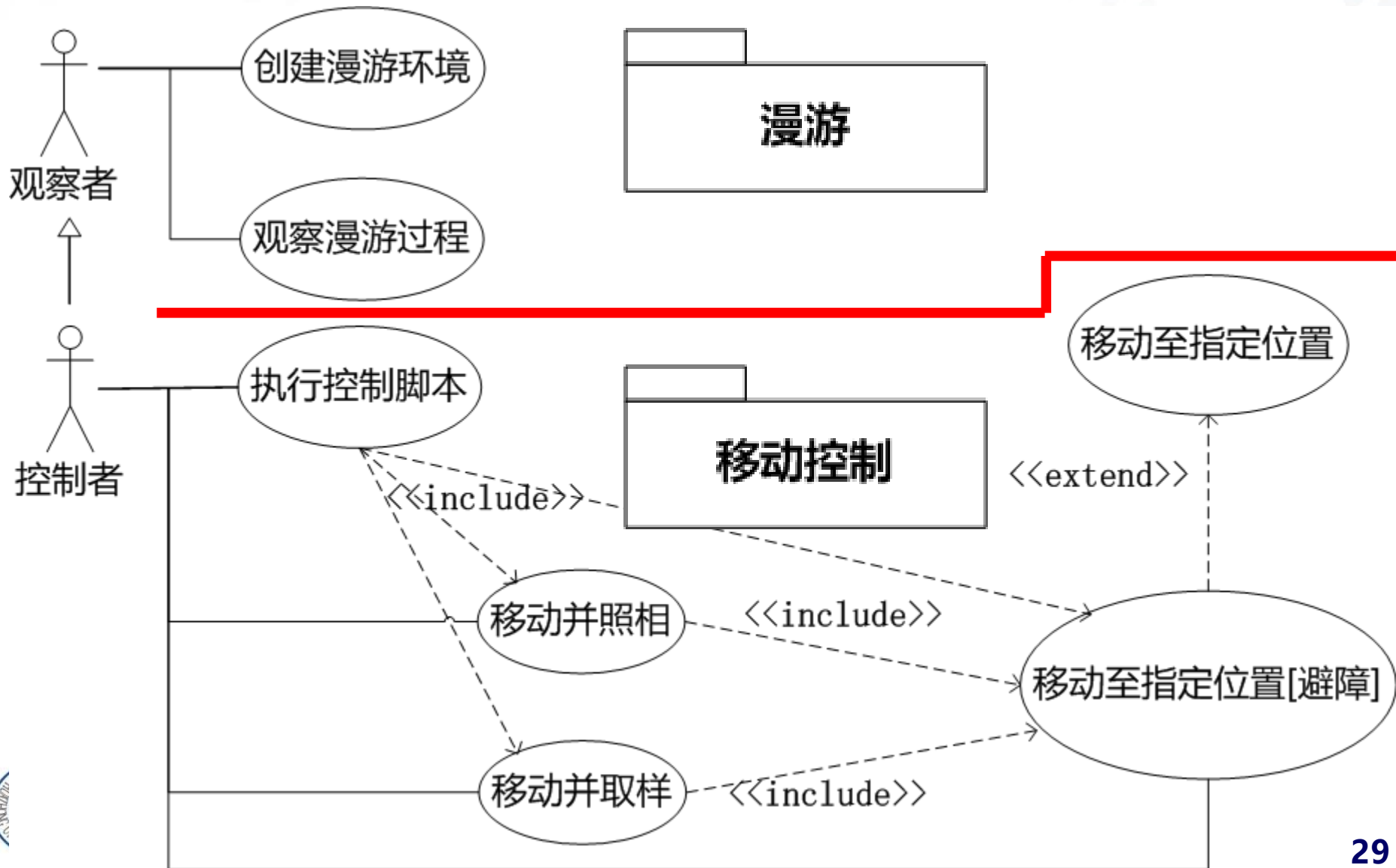
◆ 将分析模型中的职责落实到设计元素

如何“确立设计元素”？

- 方法1：对用例按照某种业务上的相关性或相似性进行分组，每组用例归为一个包/子系统，参与这些用例的实现的分析类的职责综合成为其职责

- ◆ 强内聚、松耦合原则

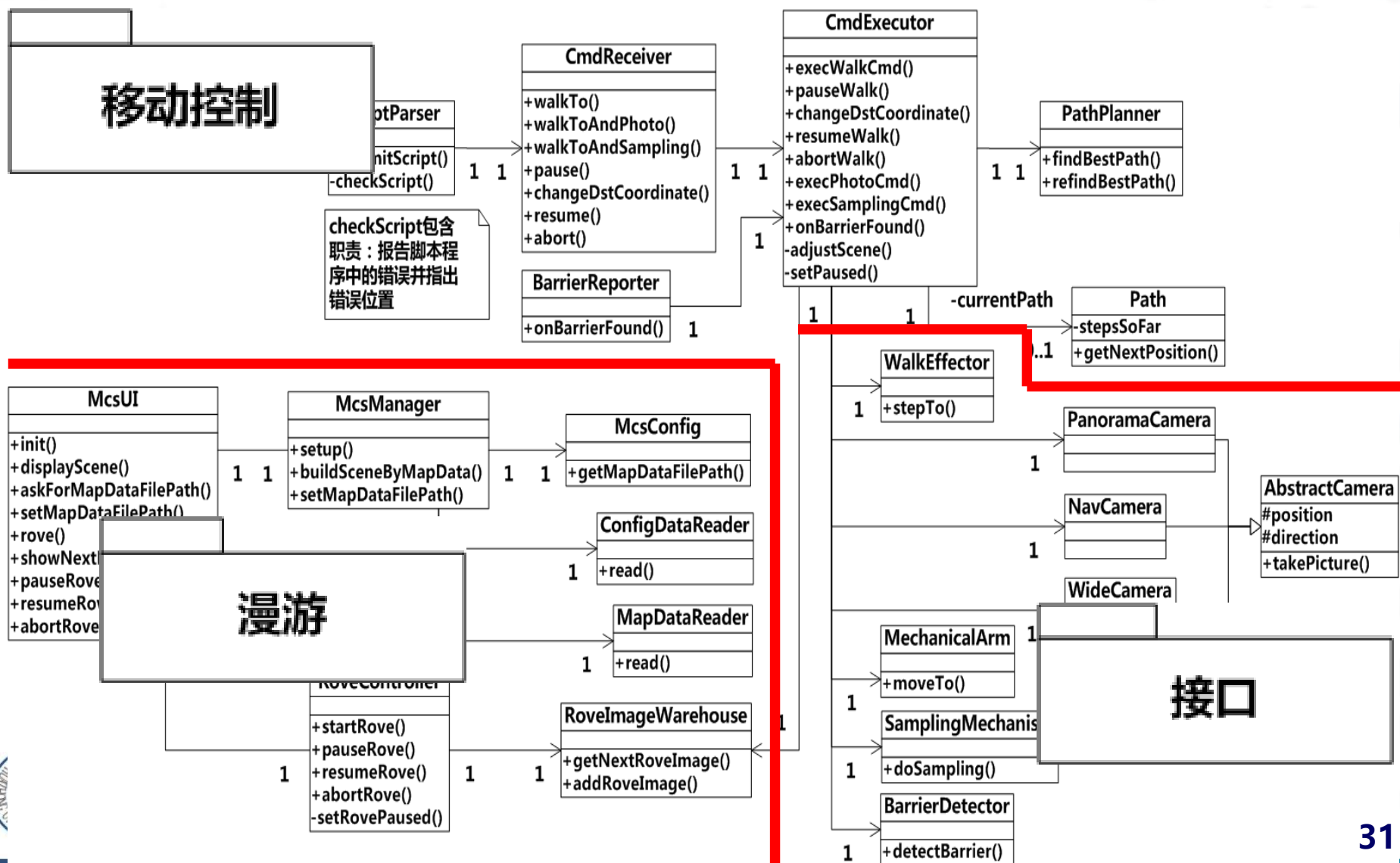
以MCS的用例图为例，确立设计元素



如何“确立设计元素”(2)

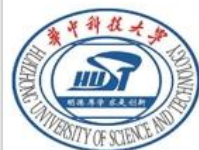
- 方法2：研究分析类图，将具有相关或者相似的业务处理职责的分析类归为一个包/子系统
 - ◆ 强内聚、松耦合原则

以MCS的分析类图为例，确立设计元素



如何“设立设计元素”(3)

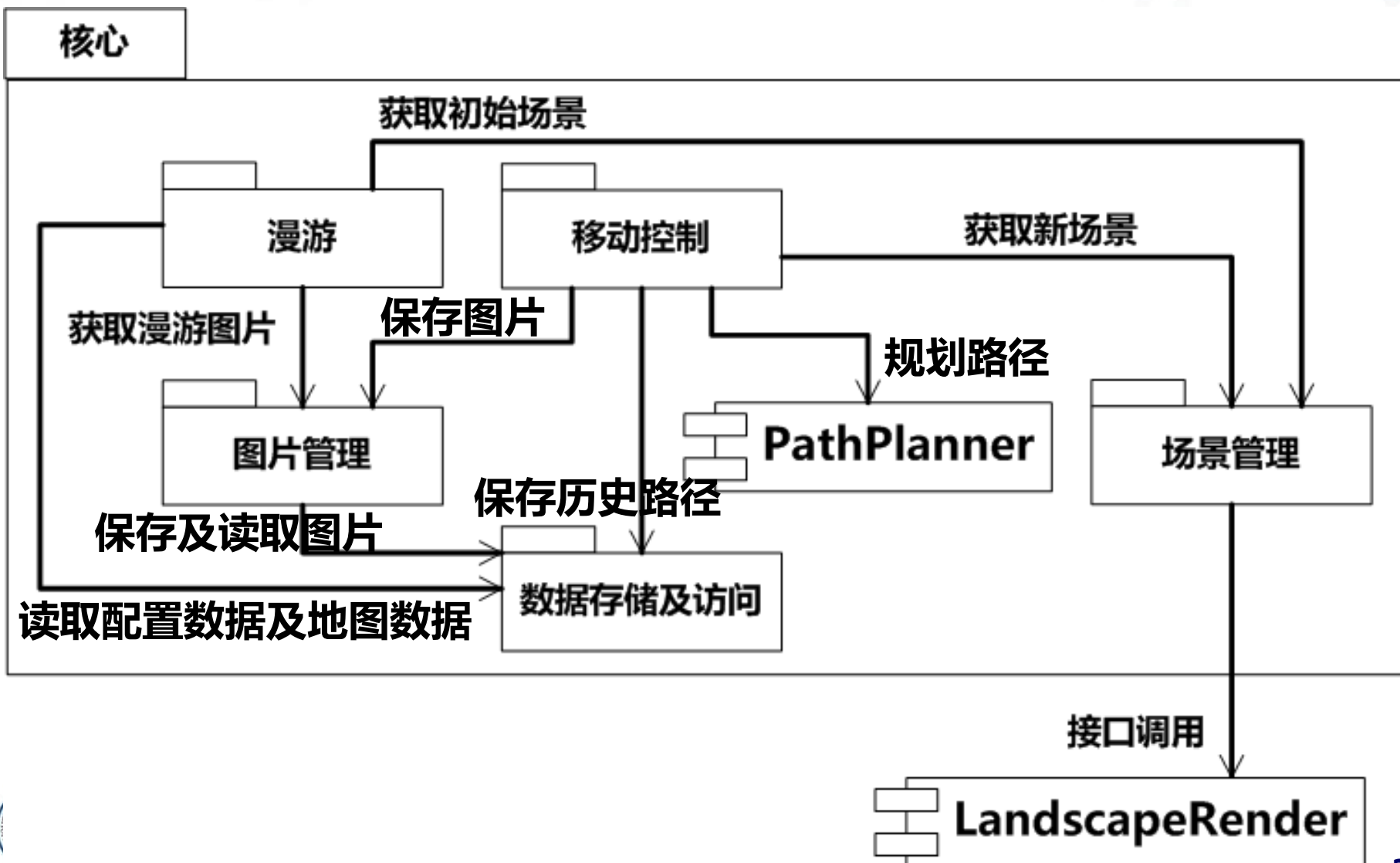
- 方法3：针对 包/子系统 或 其中某个子部分，研究其复用的必要性和可能性，必要时将其设立为 构件



以MCS为例，是否应将其中某些职责
设立为可供未来复用的构件？



MCS的体系结构(“核心”部分)演变成...

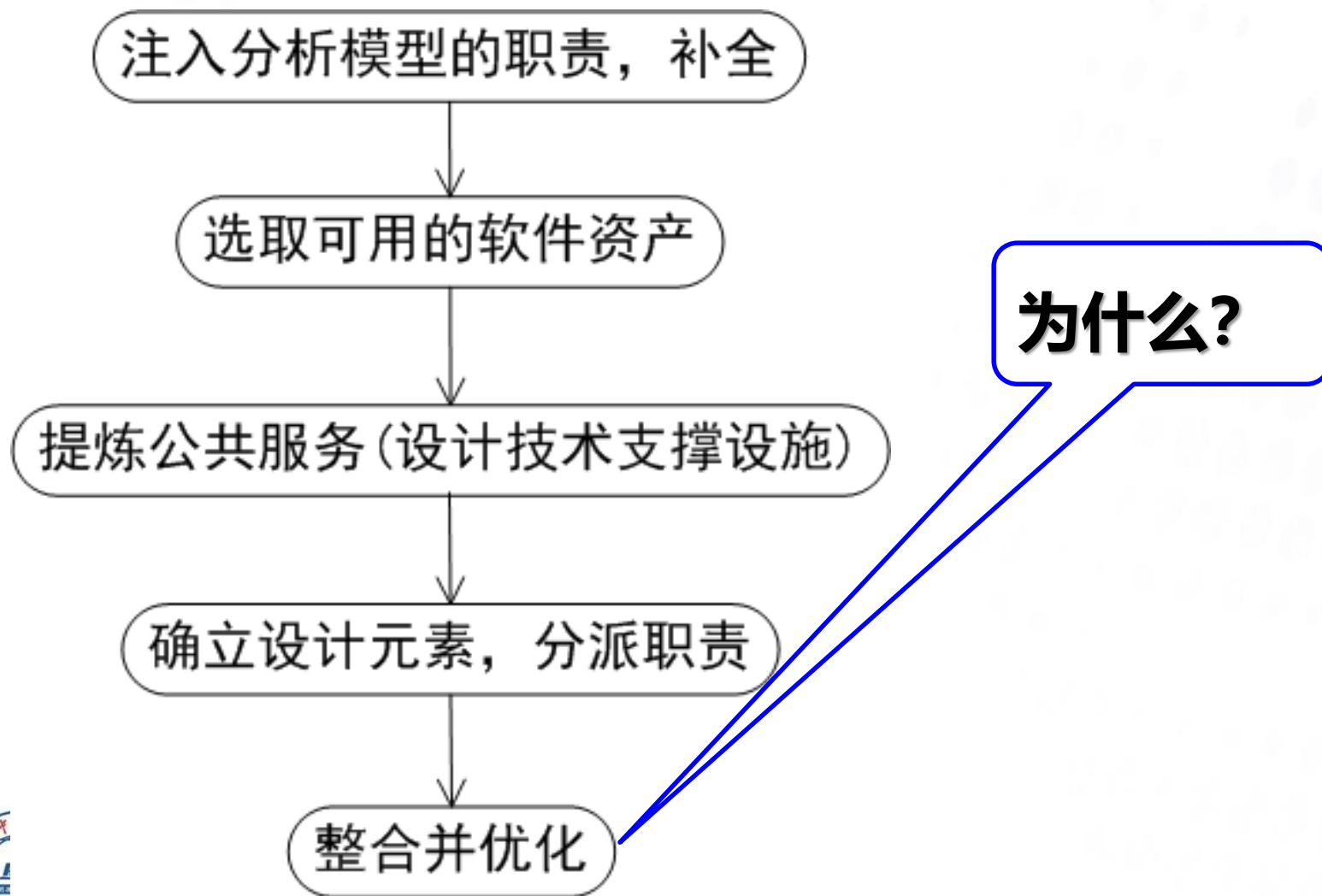


设计元素设置的合理性检查规则

- 模块内的职责具有较强的相关性或相似性
- 每个模块应负责系统中相对独立的一组功能
- 两个模块不应具有相同或相似的职责
- 在整个系统范围内，模块的规模应相对平衡，避免特别庞大或特别细小的模块
- 模块对外接口极小化，即，仅公开必要的接口函数，尽可能隐藏内部实现细节



体系结构精化的过程模型(5)



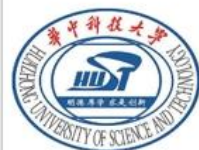
何谓“整合并优化”？

□ 关键意图：

- ◆ 整合迄今获得的所有设计元素，以协同完成所有的软件需求项
- ◆ 优化
- ◆ 构造软件体系结构的完整逻辑视图

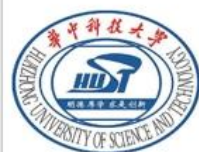
为什么必须“整合并优化”？

- 在全局的高度重新审查(review)体系结构的合理性、协调性、优化性

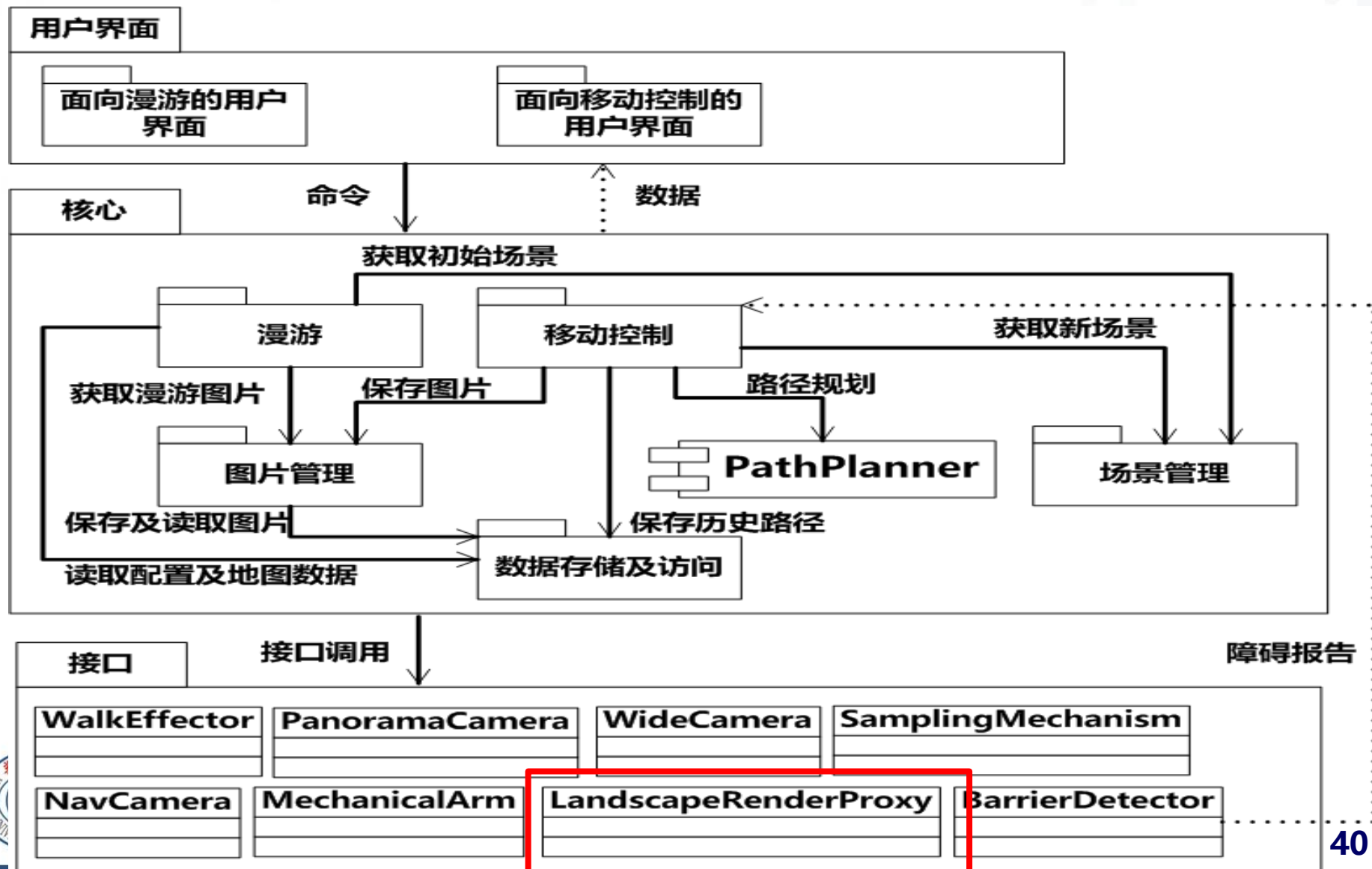


如何“整合并优化”？

- 比对服务提供者与服务请求者，检查接口定义的一致性
- 检查边界类的接口定义与外部系统（设备）接口定义的一致性
- 设计优化：
 - ◆ 提取公共职责为服务构件或服务类
 - ◆ 合并职责相同或相似的设计元素
 - ◆ 以复用为目的调整设计元素的接口
 - ◆ 调整结构以优化目标软件的质量要素，……



案例分析：精化后的MCS体系结构图



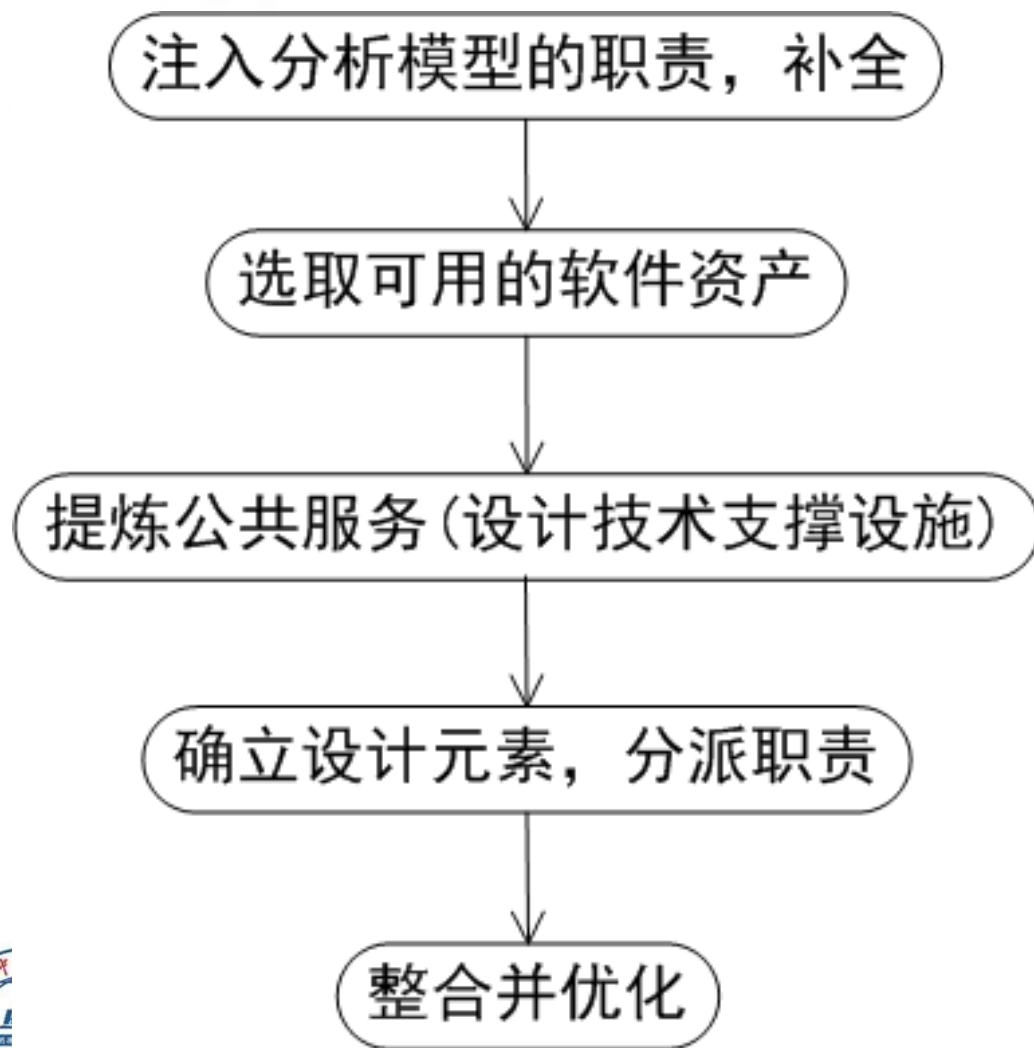
基于MCS的体系结构图， 需要定义哪些设计元素的对外接口？

- “PathPlanner” 构件
- “接口包” 中的所有边界类
- “图片管理”、“场景管理”、“数据存储及访问” 三个包的服务接口

请自行完成以上接口的定义



总结(1): 如何精化体系结构



总结(2): 如何设计软件体系结构

辨识关键需求(关键的功能需求和关键的质量需求)

创建体系结构雏形(选取适当的模式, 注入分析模型中的职责, 建立逻辑关联)

自主学习!

精化体系结构(补全, 复用已有资产, 提炼公共服务, 优化)

验证体系结构